

internlm/Intern-S1-mini · Hugging Face

Source: <https://huggingface.co/internlm/Intern-S1-mini>

[Hugging Face](#)

[Models](#)

[Datasets](#)

[Spaces](#)

[Buckets](#)

[NEW](#)

[Docs](#)

[Enterprise](#)

[Pricing](#)

[Log In](#)

[Sign Up](#)

[internlm](#)

[/](#)

[Intern-S1-mini](#)

[like](#)

[114](#)

[Follow](#)

[Intern Large Models](#)

[989](#)

[Image-Text-to-Text](#)

[Transformers](#)

[Safetensors](#)

[internsl](#)

[text-generation](#)

[conversational](#)

[custom_code](#)

[arxiv:](#)

[2508.15763](#)

[License:](#)

[apache-2.0](#)

[Model card](#)

[Files and versions](#)

[xet](#)

[Community](#)

[10](#)

[Deploy](#)

[Use this model](#)

[Intern-S1-mini](#)

[📄 Github Repo](#) • [👤 Model Collections](#) • [📄 Technical Report](#) • [🏠 Project Page](#)

[👋 join us on Discord and WeChat](#)

[Introduction](#)

We introduce Intern-S1-mini, a lightweight open-source multimodal reasoning model based on the same techniques as Intern-S1. Built upon an 8B dense language model (Qwen3) and a 0.3B Vision encoder (InternViT), Intern-S1-mini has been further pretrained on 5 trillion tokens

of multimodal data, including over 2.5 trillion scientific-domain tokens. This enables the model to retain strong general capabilities while excelling in specialized scientific domains such as interpreting chemical structures, understanding protein sequences, and planning compound synthesis routes, making Intern-S1-mini to be a capable research assistant for real-world scientific applications.

Features

Strong performance across language and vision reasoning benchmarks, especially scientific tasks.

Continuously pretrained on a massive 5T token dataset, with over 50% specialized scientific data, embedding deep domain expertise.

Dynamic tokenizer enables native understanding of molecular formulas and protein sequences.

Performance

We evaluate the Intern-S1-mini on various benchmarks including general datasets and scientific datasets. We report the performance comparison with the recent VLMS and LLMs below.

		Intern-S1-mini	Qwen3-8B	GLM-4.1V	MiMo-VL-7B-RL-2508
General	MMLU-Pro	74.78	73.7	57.1	73.93
	MMMU	72.33	N/A	69.9	70.4
	MMStar	65.2	N/A	71.5	72.9
	GPQA	65.15	62	50.32	60.35
	AIME2024	84.58	76	36.2	72.6
	AIME2025	80	67.3	32	64.4
	MathVision	51.41	N/A	53.9	54.5
	MathVista	70.3	N/A	80.7	79.4
	IFEval	81.15	85	71.53	71.4
Scientific	SFE	35.84	N/A	43.2	43.9
	Physics	28.76	N/A	28.3	28.2
	SmolInstruct	32.2	17.6	18.1	16.11
	ChemBench	76.47	61.1	56.2	66.78
	MatBench	61.55	45.24	54.3	46.9
	MicroVQA	56.62	N/A	50.2	50.96
	ProteinLMBench	58.47	59.1	58.3	59.8
	MSEarthMCQ	58.12	N/A	50.3	47.3
	XLRS-Bench	51.63	N/A	49.8	12.29

We use the OpenCompass and VLMEvalkit to evaluate all models.

Quick Start

Sampling Parameters

We recommend using the following hyperparameters to ensure better results

```
top_p = 1.0
top_k = 50
min_p = 0.0
temperature = 0.8
```

Transformers

The following provides demo code illustrating how to generate based on text and multimodal inputs.

Please use transformers>=4.55.2 to ensure the model works normally.

Text input

```
from transformers import AutoProcessor, AutoModelForCausalLM
import torch

model_name = "internlm/Intern-S1-mini"
processor = AutoProcessor.from_pretrained(model_name, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto",
torch_dtype="auto", trust_remote_code=True)

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "tell me about an interesting physical phenomenon."},
        ],
    }
]

inputs = processor.apply_chat_template(messages, add_generation_prompt=True, tokenize=True,
return_dict=True, return_tensors="pt").to(model.device, dtype=torch.bfloat16)

generate_ids = model.generate(**inputs, max_new_tokens=32768)
decoded_output = processor.decode(generate_ids[0, inputs["input_ids"].shape[1] :],
skip_special_tokens=True)
print(decoded_output)
```

Image input

```
from transformers import AutoProcessor, AutoModelForCausalLM
import torch

model_name = "internlm/Intern-S1-mini"
processor = AutoProcessor.from_pretrained(model_name, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto",
torch_dtype="auto", trust_remote_code=True)

messages = [
    {
        "role": "user",
        "content": [
            {"type": "image", "url":
"http://images.cocodataset.org/val2017/000000039769.jpg"},
            {"type": "text", "text": "Please describe the image explicitly."},
        ],
    }
]

inputs = processor.apply_chat_template(messages, add_generation_prompt=True, tokenize=True,
return_dict=True, return_tensors="pt").to(model.device, dtype=torch.bfloat16)
```

```
generate_ids = model.generate(**inputs, max_new_tokens=32768)
decoded_output = processor.decode(generate_ids[0, inputs["input_ids"].shape[1] :],
skip_special_tokens=True)
print(decoded_output)
```

Video input

Please ensure that the decord video decoding library is installed via `pip install decord`. To avoid OOM, please install `flash_attention` and use at least 2 GPUS.

```
from transformers import AutoProcessor, AutoModelForCausalLM
import torch

model_name = "internlm/Intern-S1-mini"
processor = AutoProcessor.from_pretrained(model_name, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto",
torch_dtype="auto", trust_remote_code=True)

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "video",
                "url": "https://huggingface.co/datasets/hf-internal-
testing/fixtures_videos/resolve/main/tennis.mp4",
            },
            {"type": "text", "text": "What type of shot is the man performing?"},
        ],
    }
]
```

```
inputs = processor.apply_chat_template(
    messages,
    return_tensors="pt",
    add_generation_prompt=True,
    video_load_backend="decord",
    tokenize=True,
    return_dict=True,
).to(model.device, dtype=torch.float16)
```

```
generate_ids = model.generate(**inputs, max_new_tokens=32768)
decoded_output = processor.decode(generate_ids[0, inputs["input_ids"].shape[1] :],
skip_special_tokens=True)
print(decoded_output)
```

Serving

The minimum hardware requirements for deploying Intern-S1 series models are:

Model	A100 (GPUs)	H800 (GPUs)	H100 (GPUs)	H200 (GPUs)
internlm/Intern-S1-mini	1	1	1	1
internlm/Intern-S1-mini-FP8	-	1	1	1

You can utilize one of the following LLM inference frameworks to create an OpenAI compatible server:

```
lmdeploy (>=0.9.2.post1)
lmdeploy serve api_server internlm/Intern-S1-mini --reasoning-parser intern-s1 --tool-call-
parser intern-s1
```

```
vllm (>=0.10.1)
vllm serve internlm/Intern-S1-mini --trust-remote-code
```

```
sglang
python3 -m sglang.launch_server \
    --model-path internlm/Intern-S1-mini \
    --trust-remote-code \
    --grammar-backend none
```

```
ollama for local deployment:
# install ollama
curl -fsSL https://ollama.com/install.sh | sh
# fetch model
ollama pull internlm/interns1-mini
# run model
ollama run internlm/interns1-mini
# then use openai client to call on http://localhost:11434/v1
```

Advanced Usage

Tool Calling

Many Large Language Models (LLMs) now feature Tool Calling, a powerful capability that allows them to extend their functionality by interacting with external tools and APIs. This enables models to perform tasks like fetching up-to-the-minute information, running code, or calling functions within other applications.

A key advantage for developers is that a growing number of open-source LLMs are designed to be compatible with the OpenAI API. This means you can leverage the same familiar syntax and structure from the OpenAI library to implement tool calling with these open-source models. As a result, the code demonstrated in this tutorial is versatile—it works not just with OpenAI models, but with any model that follows the same interface standard.

To illustrate how this works, let's dive into a practical code example that uses tool calling to get the latest weather forecast (based on lmdeploy api server).

```
from openai import OpenAI
import json

def get_current_temperature(location: str, unit: str = "celsius"):
    """Get current temperature at a location.

    Args:
        location: The location to get the temperature for, in the format "City, State,
        Country".
        unit: The unit to return the temperature in. Defaults to "celsius". (choices:
        ["celsius", "fahrenheit"])
```

Returns:

the temperature, the location, and the unit in a dict
"""

```
return {
    "temperature": 26.1,
    "location": location,
    "unit": unit,
}
```

```
def get_temperature_date(location: str, date: str, unit: str = "celsius"):
    """Get temperature at a location and date.
```

Args:

location: The location to get the temperature for, in the format "City, State, Country".

date: The date to get the temperature for, in the format "Year-Month-Day".

unit: The unit to return the temperature in. Defaults to "celsius". (choices: ["celsius", "fahrenheit"])

Returns:

the temperature, the location, the date and the unit in a dict
"""

```
return {
    "temperature": 25.9,
    "location": location,
    "date": date,
    "unit": unit,
}
```

```
def get_function_by_name(name):
    if name == "get_current_temperature":
        return get_current_temperature
    if name == "get_temperature_date":
        return get_temperature_date
```

```
tools = [{
    'type': 'function',
    'function': {
        'name': 'get_current_temperature',
        'description': 'Get current temperature at a location.',
        'parameters': {
            'type': 'object',
            'properties': {
                'location': {
                    'type': 'string',
                    'description': 'The location to get the temperature for, in the format \'City, State, Country\''
                },
                'unit': {
                    'type': 'string',
                    'enum': [
                        'celsius',
                        'fahrenheit'
                    ],
                    'description': 'The unit to return the temperature in. Defaults to
```

```

    \celsius\'.
    }
    },
    'required': [
        'location'
    ]
}
}, {
    'type': 'function',
    'function': {
        'name': 'get_temperature_date',
        'description': 'Get temperature at a location and date.',
        'parameters': {
            'type': 'object',
            'properties': {
                'location': {
                    'type': 'string',
                    'description': 'The location to get the temperature for, in the format
\'City, State, Country\'.
                },
                'date': {
                    'type': 'string',
                    'description': 'The date to get the temperature for, in the format
\'Year-Month-Day\'.
                },
                'unit': {
                    'type': 'string',
                    'enum': [
                        'celsius',
                        'fahrenheit'
                    ],
                    'description': 'The unit to return the temperature in. Defaults to
\'celsius\'.
                }
            },
            'required': [
                'location',
                'date'
            ]
        }
    }
}
}]

```

```

messages = [
    {'role': 'user', 'content': 'Today is 2024-11-14, What\'s the temperature in San
Francisco now? How about tomorrow?'}
]

```

```

openai_api_key = "EMPTY"
openai_api_base = "http://0.0.0.0:23333/v1"
client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

```

```

model_name = client.models.list().data[0].id
response = client.chat.completions.create(
    model=model_name,
    messages=messages,
    max_tokens=32768,
    temperature=0.8,
    top_p=0.8,
    stream=False,
    extra_body=dict(spaces_between_special_tokens=False, enable_thinking=False),
    tools=tools)
print(response.choices[0].message)
messages.append(response.choices[0].message)

for tool_call in response.choices[0].message.tool_calls:
    tool_call_args = json.loads(tool_call.function.arguments)
    tool_call_result = get_function_by_name(tool_call.function.name)(**tool_call_args)
    tool_call_result = json.dumps(tool_call_result, ensure_ascii=False)
    messages.append({
        'role': 'tool',
        'name': tool_call.function.name,
        'content': tool_call_result,
        'tool_call_id': tool_call.id
    })

response = client.chat.completions.create(
    model=model_name,
    messages=messages,
    temperature=0.8,
    top_p=0.8,
    stream=False,
    extra_body=dict(spaces_between_special_tokens=False, enable_thinking=False),
    tools=tools)
print(response.choices[0].message.content)

```

Switching Between Thinking and Non-Thinking Modes

Intern-S1-mini enables thinking mode by default, enhancing the model's reasoning capabilities to generate higher-quality responses. This feature can be disabled by setting `enable_thinking=False` in `tokenizer.apply_chat_template`

```

text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=False # think mode indicator
)

```

With LMDeploy serving Intern-S1-mini models, you can dynamically control the thinking mode by adjusting the `enable_thinking` parameter in your requests.

```

from openai import OpenAI
import json

```

```

messages = [
{

```



```

        'role': 'user',
        'content': 'who are you'
    }, {
        'role': 'assistant',
        'content': 'I am an AI'
    }, {
        'role': 'user',
        'content': 'AGI is?'
    }
]]

openai_api_key = "EMPTY"
openai_api_base = "http://0.0.0.0:23333/v1"
client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)
model_name = client.models.list().data[0].id

response = client.chat.completions.create(
    model=model_name,
    messages=messages,
    temperature=0.8,
    top_p=0.8,
    max_tokens=2048,
    extra_body={
        "enable_thinking": False,
    }
)
print(json.dumps(response.model_dump(), indent=2, ensure_ascii=False))

```

For vllm and sglang users, configure this through,

```

extra_body={
    "chat_template_kwargs": {"enable_thinking": False}
}

```

Fine-tuning

See this [documentation](#) for more details.

Citation

If you find this work useful, feel free to give us a cite.

```

@misc{bai2025interns1scientificmultimodalfoundation,
  title={Intern-S1: A Scientific Multimodal Foundation Model},
  author={Lei Bai and Zhongrui Cai and Maosong Cao and Weihao Cao and Chiyu Chen and Haojiong Chen and Kai Chen and Pengcheng Chen and Ying Chen and Yongkang Chen and Yu Cheng and Yu Cheng and Pei Chu and Tao Chu and Erfei Cui and Ganqu Cui and Long Cui and Ziyun Cui and Nianchen Deng and Ning Ding and Nanqin Dong and Peijie Dong and Shihan Dou and Sinan Du and Haodong Duan and Caihua Fan and Ben Gao and Changjiang Gao and Jianfei Gao and Songyang Gao and Yang Gao and Zhangwei Gao and Jiaye Ge and Qiming Ge and Lixin Gu and Yuzhe Gu and Aijia Guo and Qipeng Guo and Xu Guo and Conghui He and Junjun He and Yili Hong and Siyuan Hou and Caiyu Hu and Hanglei Hu and Jucheng Hu and Ming Hu and Zhouqi Hua and Haian Huang and Junhao Huang and Xu Huang and Zixian Huang and Zhe Jiang and Lingkai Kong and Linyang Li

```

and Peiji Li and Pengze Li and Shuaibin Li and Tianbin Li and Wei Li and Yuqiang Li and Dahua Lin and Junyao Lin and Tianyi Lin and Zhishan Lin and Hongwei Liu and Jiangning Liu and Jiyao Liu and Junnan Liu and Kai Liu and Kaiwen Liu and Kuikun Liu and Shichun Liu and Shudong Liu and Wei Liu and Xinyao Liu and Yuhong Liu and Zhan Liu and Yinquan Lu and Haijun Lv and Hongxia Lv and Huijie Lv and Qidang Lv and Ying Lv and Chengqi Lyu and Chenglong Ma and Jianpeng Ma and Ren Ma and Runmin Ma and Runyuan Ma and Xinzhu Ma and Yichuan Ma and Zihan Ma and Sixuan Mi and Junzhi Ning and Wenchang Ning and Xinle Pang and Jiahui Peng and Runyu Peng and Yu Qiao and Jiantao Qiu and Xiaoye Qu and Yuan Qu and Yuchen Ren and Fukai Shang and Wenqi Shao and Junhao Shen and Shuaike Shen and Chunfeng Song and Demin Song and Diping Song and Chenlin Su and Weijie Su and Weigao Sun and Yu Sun and Qian Tan and Cheng Tang and Huanze Tang and Kexian Tang and Shixiang Tang and Jian Tong and Aoran Wang and Bin Wang and Dong Wang and Lintao Wang and Rui Wang and Weiyun Wang and Wenhai Wang and Yi Wang and Ziyi Wang and Ling-I Wu and Wen Wu and Yue Wu and Zijian Wu and Linchen Xiao and Shuhao Xing and Chao Xu and Huihui Xu and Jun Xu and Ruiliang Xu and Wanghan Xu and GanLin Yang and Yuming Yang and Haochen Ye and Jin Ye and Shenglong Ye and Jia Yu and Jiashuo Yu and Jing Yu and Fei Yuan and Bo Zhang and Chao Zhang and Chen Zhang and Hongjie Zhang and Jin Zhang and Qiaosheng Zhang and Qiuyinzhe Zhang and Songyang Zhang and Taolin Zhang and Wenlong Zhang and Wenwei Zhang and Yechen Zhang and Ziyang Zhang and Haiteng Zhao and Qian Zhao and Xiangyu Zhao and Xiangyu Zhao and Bowen Zhou and Dongzhan Zhou and Peiheng Zhou and Yuhao Zhou and Yunhua Zhou and Dongsheng Zhu and Lin Zhu and Yicheng Zou},

```

    year={2025},
    eprint={2508.15763},
    archivePrefix={arXiv},
    primaryClass={cs.LG},
    url={https://arxiv.org/abs/2508.15763},
}

```

Downloads last month

2,403

Safetensors

Model size

9B params

Tensor type

BF16

Chat template

Files info

Inference Providers

NEW

Image-Text-to-Text

This model isn't deployed by any Inference Provider.



1

Ask for provider support

Model tree for

internlm/Intern-S1-mini

Finetunes

2 models

Quantizations

5 models

Collection including

internlm/Intern-S1-mini

Intern-S1

Collection

10 items

-

Updated 14 days ago

-

32

Paper for

internlm/Intern-S1-mini

Intern-S1: A Scientific Multimodal Foundation Model

Paper

-

2508.15763

-

Published Aug 21, 2025

-

273

System theme

TOS

Privacy

About

Careers

Models

Datasets

Spaces

Pricing

Docs